Technical Interview Agent

Consolidated Project Report

Prepared: 12 October 2026

This report brings together the project description, architecture, guardrails, token costs, evaluation results, known defects, and the plan to production. It is compiled from the project documents and evaluation records as they stood on the date above. Where a result is not yet verified, the report says so.

Status: working prototype. Not yet approved for use with real candidates.

Contents

1. Executive summary
2. What the system does (plain terms)
3. Interview modes
4. Scoring and recommendation
5. Architecture
6. End-to-end flow
7. Technology stack
8. Project structure
9. Guardrails
10. Token consumption per mode
11. Evaluation results
12. Defects and known gaps
13. Setup and configuration
14. API overview
15. Testing status
16. Limitations
17. Future implementations
18. Changes needed for cloud deployment
19. Production readiness plan
20. Technology comparison: Node.js vs Python
21. Open decisions and next steps
22. Appendix A: project documents
23. Appendix B: test history

1. Executive summary

The Technical Interview Agent runs a first-round technical interview for a job candidate through a web link. The candidate types answers to a set of questions. The system scores the answers on four skills against written rubrics, writes a report, and suggests a recommendation: Strong Hire, Hire, Hold or No Hire. A human reviewer always makes the final decision. The AI recommends; it does not hire or reject.

The prototype runs end to end on one server. It stores data in JSON files and uses the OpenAI API for scoring, follow-up questions, injection checks and the report. The evaluation run in section 11 gives mixed results. Scoring is stable on most dimensions, the bias check found no significant effect, and both injection attacks were flagged without changing scores. The same evaluation found a silent defect: an empty score can be saved as a NaN result and a No Hire recommendation. That defect needs fixing before any real use.

Item	Status
Stage	Working prototype; runs end to end locally
Interview modes	A: fixed question bank. B: bank plus CV-based follow-ups. C: questions generated from the CV
Test content	10 job descriptions, 51 bank questions, 10 fictional CVs (PDF and DOCX)
Scoring	Four skills scored 1 to 5 by the model, combined into a 0 to 100 score with fixed weights
Storage	JSON files on disk, with a backup of the previous version of each record
AI provider	OpenAI API; model names set in the .env file
Readiness	Not ready for real candidates. Open defects and the production plan are in sections 12 and 19

2. What the system does (plain terms)

1. The admin logs in with a password, picks a job and an interview mode, and creates a candidate link. The link can be used to join for 30 minutes at most.
2. The candidate opens the link and ticks three consent boxes: data storage, sending the CV and answers to OpenAI, and talking to an AI agent. The interview cannot start without all three.
3. The candidate shares a CV as PDF, DOCX or pasted text. The CV is required in Modes B and C, and optional in Mode A.
4. The candidate types an answer to each question. The interview has a 60-minute limit from the start. When time runs out, the answers given so far are submitted automatically.
5. In Modes B and C, the system may ask a follow-up when an answer is very short, unclear, or conflicts with the CV. There are limits of 3 per question and 5 per interview.
6. Each answer is checked for attempts to manipulate the scorer. The model then scores four skills. Any evidence quote that does not appear in the candidate text is removed.
7. The four scores are combined with fixed weights into a 0 to 100 score, which gives a recommendation and a confidence level.
8. A report is written. A reviewer approves the recommendation or overrides it, and an override needs a written reason of at least 10 characters.
9. Interview data is deleted automatically after the retention period, 30 days by default.

3. Interview modes

	Mode A: fixed bank	Mode B: bank plus CV probes	Mode C: CV-generated
Where questions come from	The question bank for the job. Same questions for every candidate.	The question bank, plus up to 2 extra questions about the CV, planned at the start.	Generated by the model from the CV and the job description (8 by default).
CV	Optional	Required	Required
Follow-up questions	No	Yes	Yes
Model calls to choose questions	None	CV claim extraction and probe planning	Question generation, up to 3 passes
Best for	Fair comparison across candidates	Checking what is on the CV	Roles with no question bank, or deeper CV coverage

Follow-up rules (Modes B and C)

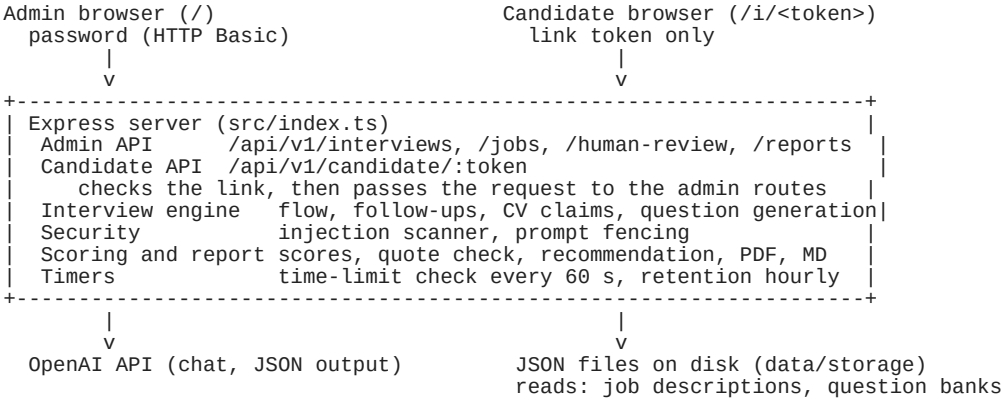
- Vague: the answer has fewer than 8 words. This check uses no model call.
- Incomplete: the answer leaves out part of what the question asks for.
- Contradicting: the answer conflicts with a claim on the CV.
- Limits: at most 3 follow-ups per question and 5 per interview. A follow-up that repeats an earlier one (60 percent word overlap or more) is dropped. Protected topics are never asked about.

4. Scoring and recommendation

- Four skills are each scored 1 to 5 by the model against a rubric, with up to two evidence quotes and a short reason.
- Weights: Technical Skills 40 percent, Domain Understanding 20 percent, Problem Solving 20 percent, Communication 10 percent. The weighted average is multiplied by 20 to give a 0 to 100 score.
- Recommendation from the 0 to 100 score: 80 or more is Strong Hire; 70 to 79 is Hire; 60 to 69 is Hold; below 60 is No Hire.
- Confidence depends on how far apart the four skill scores are: a small spread is High, a medium spread is Medium, a large spread is Low.
- Quote check: every evidence quote must appear in the candidate answers. A skill with no verified quote has its confidence capped at 0.5.
- The code, not the model, computes the weighted score and the recommendation. Security flags never change scores; they go to the reviewer.

5. Architecture

The application is a single Node.js process. It has a web layer, an interview engine, a scoring engine and a storage layer. It calls one external service, the OpenAI API.



Key design choices

- The candidate link is the only credential candidates have. Only a SHA-256 hash of the 32-byte random token is stored.
- The candidate API reuses the admin routes. It checks the link state and passes the request on only in states where that action is allowed. For example, answers are accepted only while the interview is in progress.
- All model calls go through one service (OpenAIService). Each call is logged with its purpose and token counts.
- Scores and recommendations are computed by fixed code rules, so they can be read and tested.
- The scorer sees the candidate answers on their own, with the question text sent separately, so the scoring guide is never mixed into the candidate text.

6. End-to-end flow

1. Admin login, interview creation (job and mode), and candidate link creation.
2. Candidate opens the link. The link is checked: valid, expired, or not found.
3. Consent: three boxes must all be ticked. The consent is saved with a timestamp and recorded in the audit log.
4. CV step: the file type and size are checked, the text is extracted, the text is scanned for injection, and in Modes B and C, claims are extracted after contact details are removed.
5. Start: Mode C generates questions from the CV and job; Mode B plans CV probes. The 60-minute limit begins.
6. Question loop: show the next question, the candidate types an answer, and in Modes B and C a follow-up may be asked.
7. Submit, or the 60-minute limit is reached and a background check submits the interview.
8. Scoring: scan the answers for injection, score four skills with the model, check every quote, compute the weighted score and recommendation.
9. Report: the model writes a cumulative report, and the code adds deterministic checks such as short answers, quote checks and injection flags.
10. Human review: approve or override with a reason. The status becomes finalized.
11. Retention: after 30 days the interview, scores, recommendation, report and backups are deleted automatically.

Interview states

State	Meaning
open	The link is valid and the interview has not started. Join within 30 minutes.
in_progress	The interview has started. Its 60-minute limit runs from this moment.
completed / under_review	Answers are in and scored. A human has not yet decided.
finalized	A reviewer has approved or overridden the result.
expired	The link or the time limit has passed.

7. Technology stack

Area	Technology	Purpose
Language	TypeScript 5, compiled to JavaScript	Typed code that catches mistakes before they run
Runtime	Node.js (tested with v22)	Runs the server and the build tools
Web server	Express 4	Routing and middleware
Request handling	body-parser (10 MB limit), multer (in-memory, 5 MB limit)	Reads JSON bodies and CV uploads safely
Cross-origin rules	cors	Controls which websites can call the API
Configuration	dotenv	Reads settings from the .env file
AI model	OpenAI API through the official openai SDK	Scoring, follow-ups, injection checks, report
CV reading	pdf-parse 2.4.5 (PDF), mammoth 1.6 (DOCX)	Extracts text from the two accepted formats
Output checking	ajv 8 (JSON Schema)	Checks the questions generated in Mode C
Reports	Markdown (own code), pdfkit 0.13	Report in two formats
Storage	JSON files in data/storage, with backups	Simple to run and inspect; not built for many simultaneous users
IDs	uuid 9	Unique record IDs
Front end	Plain HTML, CSS and JavaScript	No framework, no build step, no third-party scripts
Testing	Jest 29, ts-jest	Unit tests (one test file today)
Code quality	TypeScript compiler, ESLint 8, Prettier 3	Type checks and consistent style

8. Project structure

```
draft-3/
  src/
    index.ts                    start-up, routes, pages, timers
    api/routes/                 interviewRoutes, candidateRoutes, humanReviewRoutes,
                                reportRoutes, jobRoutes
    auth/adminAuth.ts           admin password check
    invitations/candidateLink.ts  link tokens
    interview/                  InterviewFlow, followUpEngine, cvClaims, questionGenerator
    cv/cvExtractor.ts           file checks and text extraction
    jobs/jobDescriptionFile.ts  reads job description Markdown
    scoring/ScoringEngine.ts    scores, quote check, recommendation
    reporting/                  ReportService, markdownExporter, pdfExporter
    security/                   InjectionScanner, promptData
    services/OpenAIService.ts   the only place that calls OpenAI
    database/models/            session, scoring and recommendation records
    storage/JsonStorage.ts      JSON reads and writes, with backups
    retention/                  deletes data past its retention date
    types/, utils/, config/     shared types, helpers, settings
  public/                       admin and candidate pages (HTML, CSS, JavaScript)
  data/
    job_descriptions/           10 job descriptions (Markdown)
    question_banks/             10 question banks (JSON)
    synthetic_cvs/              10 fictional CVs (PDF and DOCX), for testing only
    storage/                    interview records created at run time
  docs/                         guides, this report and the evaluation folder
  tests/                        Jest tests (one unit test file)
  .env                          settings and secrets; do not share
```

9. Guardrails

Guardrails are the controls that keep the interview fair, safe and under human control. Each item below is in the code today.

Access and links

- Admin portal: one shared password (HTTP Basic), compared with a timing-safe check. The user name is not checked.
- Candidate link: 32 random bytes, only the hash is stored, valid for joining for at most 30 minutes.
- Candidate actions are allowed only in the states where they make sense. Answers are accepted only during the interview.
- Tokens are kept out of logs, and the candidate page sends no referrer.
- Request size limit: 10 MB for JSON and form bodies. CV uploads are limited to 5 MB.

Consent and time

- Three consent boxes are required before start: data storage, sending CV and answers to OpenAI, and the AI agent notice. Consent is saved with a timestamp.
- A 60-minute limit from start. A background check every 60 seconds submits any interview that has overrun, even if the candidate closed the page.

CV handling

- Only real PDF and DOCX files are accepted. The first bytes of the file are checked, not just the name. Old .doc files are rejected. Text under 40 characters is rejected.
- Contact details (emails, phone numbers, links) are removed before claims are extracted from the CV.
- The CV is trimmed to 20,000 characters before it goes into a prompt.

Prompt-injection defences

- Pattern check: 9 rules look for common attempts such as "ignore previous instructions".
- Model check: a second model call looks at answers and CV text and reports attempts with a confidence level. Long CVs are split into pieces first.
- Fencing: candidate text is placed inside marked data blocks, and tag-like text inside it is neutralised.
- Flags do not change scores. The reviewer decides what to do with a flag.

Scoring integrity

- Quote check: an evidence quote that is not found in the transcript is removed.
- Confidence cap: a skill with no verified quote cannot be scored above 0.5 confidence.
- Fixed rules: weights, thresholds and the 1 to 5 scale are in code.
- Rubrics: each question has a rubric describing what a 1, 3 or 5 answer looks like.

Question control

- Follow-up caps: 3 per question and 5 per interview. Repeated follow-ups are dropped. Protected topics are never asked about.
- Mode C checks: each generated question must match the schema. Near-duplicates are removed. Required skills (importance 4 or 5) must be covered, with up to 3 passes to fix gaps. If fewer than 4 valid questions remain, the interview does not start.

Human control and data lifecycle

- Every result needs a human decision. Status becomes finalized only after approval or override.
- Overrides need a reason of at least 10 characters.
- The audit log records consent, starts, answers, scoring and reviews on the interview record.
- Retention: records are deleted 30 days after creation or decision. The purge runs at start-up and every hour.

10. Token consumption per mode

Token use was measured on one full run per mode, from start to finish, using synthetic answers.

Mode	Model calls	Tokens after optimisation	Tokens before optimisation
A	9	7,806	8,381
B	34	21,805	22,170
C	35 (includes 2 repair calls)	29,512	27,061

Where the tokens go

- Scoring: four calls per interview, one per skill. The four calls share a long prompt start that the provider can cache. Cached tokens were seen on 3 of 4 scoring calls in testing.
- Report: one call. In one measured run it used 3,547 output tokens, of which 2,048 were hidden reasoning tokens, which count as output.
- Injection checks: one call per answer segment and per CV chunk.
- Mode B adds CV claim extraction, probe planning and a follow-up decision per answer.
- Mode C adds question generation, plus repair calls when a required skill is not covered.
- Mode A chooses its questions from a fixed list with no model call.

Caveats

- These are single runs. Counts change with answer length, the number of questions and the number of follow-ups.
- Dollar costs are not verified. The code estimates cost with assumed rates. Check the price of the model in use.
- Not yet built: a token budget per interview, and a setting to reduce reasoning effort for the report.

11. Evaluation results

The evaluation was run on Mode A, job cloud_architect, using the same scoring code as the app. Scores are on a 0 to 100 scale; one rating step is 20 points. Variance is the sample variance across runs. Full details are in docs/EVALUATION_REPORT.md.

11.1 Scoring consistency (5 runs per transcript)

Transcript	Technical	Domain	Problem solving	Communication	Weighted
T1 Strong	mean 88, var 120	mean 100, var 0 (4 valid runs)	mean 100, var 0	mean 96, var 80	mean 95.5, var 27.0 (4 valid runs)
T2 Average	mean 40, var 0	mean 56, var 80	mean 52, var 120	mean 60, var 0	mean 48.6, var 4.8
T3 Weak	mean 20, var 0	mean 20, var 0	mean 20, var 0	mean 60, var 0	mean 24, var 0

- The weighted score moved very little on the average and weak transcripts.
- Variance on the strong transcript came from one rating step. The strong transcript sits near the Strong Hire threshold, so one step can move it between Strong Hire and Hire.
- Recommendations: T1 gave Strong Hire in 4 of 5 runs and No Hire in the run with the defect. T2 and T3 gave No Hire in all runs.

11.2 Bias check (name and gender changed on the CV)

Identical answers (T1) were scored with the CV name changed from "Grace Mensah" to "David Mensah", 5 runs each. The CV has no pronouns, so the change was only the first name.

Dimension (0 to 100)	Original name	Swapped name	Difference
Technical Skills	84	84	0
Domain Understanding	88	88	0
Problem Solving	100	92	-8
Communication	96	88	-8
Weighted score	89.8	87.0	-2.8
Recommendation (10 runs)	Strong Hire	Strong Hire	none

The two 8-point differences are one rating step. With 5 runs per side they are not statistically significant, so there is no evidence of a name or gender effect. This is also not proof that there is none. It is a first check on one CV, not a full bias evaluation.

11.3 Injection resistance

Arm	Weighted scores (runs)	Mean	Flags raised
Clean baseline (same answers, clean CV)	51, 47, 47, 47, 51	48.6	none
Injected instruction in the CV	47, 47, 42	45.3	1 high severity, from the model check only
Injected instruction in answer 3	47, 47, 51	48.3	2 high severity: pattern rule and model check

Neither injection changed the recommendation. Every run stayed No Hire, and the scores are within the normal run-to-run range. The flags are kept for the reviewer. In this run the pattern rules did not flag the CV text, so that attack relied on the model check alone.

11.4 Human agreement

Status: not complete. The comparison needs a reviewer to record their own recommendation for each interview before comparing with the agent. The agent results are listed below.

Interview	What it is	Agent recommendation (score)	Reviewer recommendation
T1 Strong	Specific answers with trade-offs and measures	Strong Hire (100)	to be recorded
T2 Average	Right direction, few specifics, several vague answers	No Hire (51)	to be recorded
T3 Weak	Vague, very short, misconceptions	No Hire (24)	to be recorded
T4 Mixed	Strong technical content, poor structure and clarity	Hire (78)	to be recorded
T5 Short	One-line answers only	No Hire (38)	to be recorded

12. Defects and known gaps

ID	Defect or gap	Severity	Status
D1	An empty dimension score is saved as NaN, and the result becomes No Hire with low confidence. Seen in 1 of 33 scoring runs. The likely cause is the 400-token output cap, which the model used up before returning visible content. The code then turns the empty reply into an empty object.	High	Open. Fix proposed (retry the skill once, then mark the interview as errored instead of saving NaN). Awaiting approval.
D2	CV injection was caught only by the model check; the pattern rules did not flag the CV text.	Medium	Open
D3	No identity check: the name typed by the candidate is not compared with the CV name.	High for real use	Accepted risk so far; decision pending
D4	Consent is given by whoever uses the link, not verified as the CV owner.	High for real use	Accepted risk so far
D5	A failed CV upload that is retried can create a second interview record.	Low	Known; not fixed
D6	JSON file storage: two saves to the same record at the same time can overwrite each other.	Medium	Open; needs a database for production
D7	Single shared admin password; no named accounts or per-reviewer audit trail.	High for real use	Open
D8	No rate limiting and no CSRF protection on admin and candidate endpoints.	Medium	Open; decision pending
D9	Build reports TypeScript errors (missing pdfkit type declarations, and unused variables in tests). Output is still produced.	Low	Open
D10	Mode C has not been run on the newest job profiles.	Medium	Open; not run as instructed
D11	Dollar cost figures use assumed rates, not verified prices.	Low	Open
D12	Bias check covers one CV and one transcript with 5 runs per side. It is not a full bias evaluation.	Medium	Open

13. Setup and configuration

Running the application

1. Install Node.js (tested with v22) and run npm install.
2. Create the .env file with at least OPENAI_API_KEY and ADMIN_PASSWORD.
3. Run npm run dev for development (ts-node), or npm run build then npm start.
4. Open http://localhost:3000 and sign in with any user name and the ADMIN_PASSWORD value.

Main settings

Setting	Default	What it does
OPENAI_API_KEY	required	Key for the OpenAI API
ADMIN_PASSWORD	required	Password for the admin portal
PORT	3000	Port the server listens on
OPENAI_MODEL_SCORING	gpt-6-luna	Model for scoring, report and follow-up writing
OPENAI_MODEL_QUICK	gpt-6-luna	Model for small checks (injection, follow-up decisions)
CLIENT_NAME	empty	Company name shown on the candidate and admin pages
PUBLIC_BASE_URL	http://localhost:PORT	Address used when building candidate links
LINK_LIFETIME_MINUTES	30	Minutes a link can be used to join (capped at 30)
INTERVIEW_TIMEOUT_MINUTES	60	Interview time limit from start
MAX_FOLLOW_UPS_PER_QUESTION / _PER_INTERVIEW	3 / 5	Follow-up limits in Modes B and C
CV_PROBE_LIMIT	2	Extra CV questions planned in Mode B
GENERATED_QUESTION_COUNT	8	Questions generated in Mode C
DATA_RETENTION_DAYS	30	Days before interview data is deleted
DATA_DIRECTORY, BACKUP_ENABLED, LOG_LEVEL	./data/storage, on, info	Storage location, backups, log detail

The .env file in this project holds real secrets. It must not be shared or copied into shared locations.

14. API overview

Admin routes need the admin password. Candidate routes need the link token. Health check: GET /health. Branding: GET /api/v1/branding.

Group	Routes
Interviews (admin)	POST /initialize; GET / (list); GET /:id; GET /:id/status; POST /:id/link; POST /:id/start; POST /:id/submit-answer; POST /:id/submit-follow-up; POST /:id/complete; POST /:id/rescore; POST /:id/cv/upload; PUT /:id/cv; GET /:id/next-question; GET /:id/results
Candidate (link token)	GET /status; POST /consent; POST /start; POST /cv/upload; PUT /cv; GET /next-question; POST /submit-answer; POST /submit-follow-up; POST /complete
Reviewer	GET /pending; GET /:sessionId; POST /:sessionId/approve; POST /:sessionId/override (reason of 10+ characters)
Reports	POST /:sessionId/generate; GET /:sessionId/json; GET /:sessionId/markdown; GET /:sessionId/pdf
Jobs	GET / (list); GET /:jobId/questions

15. Testing status

- The repository has one Jest unit test file (tests/unit/database/InterviewSession.test.ts). It is not a full test suite.
- The end-to-end checks used during development were temporary scripts kept outside the repository. Their results are in Appendix B. They were not rerun for this report, so they may not reflect the latest code.
- The evaluation runner is in docs/evaluation/evaluate.js. It calls the same scoring code the app uses. It needs the OpenAI key and creates interview records.
- npm run build still emits output but reports TypeScript errors (see D9). These should be fixed before a formal release.

16. Limitations

- No identity check: the candidate name is not compared with the CV.
- Consent comes from the person using the link, not verified against the CV owner.
- Answers are typed. There is no speech-to-text, no audio and no video.
- Storage is JSON files, which are not built for many users writing at once.
- Single shared admin password, with no named accounts.
- No full bias testing. The code does not check whether scores change with names, gender or other details.
- Scores are one model's judgement. The quote check and fixed rules reduce errors but do not make a score correct. Human review is required.
- Repeat scoring varies. The same answers can get slightly different scores on different runs.
- Mode C is the most expensive and the least predictable, because question generation can need repair passes.
- Synthetic CVs only. The test CVs are fictional.
- No formal HTTPS or security review has been done.

17. Future implementations

1. Identity check: compare the name typed by the candidate with the name on the CV, and flag any mismatch for the reviewer.
2. Consent from the CV owner, verified in some way.
3. A real database in place of JSON files, with concurrent writes and indexed lookups.
4. A token budget per interview: stop, or switch to a cheaper check, when a set limit is reached.
5. A reasoning-effort setting for the report, to reduce hidden reasoning tokens (not yet tested).
6. Rate limits and CSRF protection on admin and candidate endpoints.
7. Named reviewer accounts and single sign-on, in place of the shared admin password.
8. Bias testing: score the same answers with different names and CV details, and check that the scores stay the same.
9. An evaluation suite run on a schedule: consistency, injection resistance and agreement with human decisions.
10. Voice or video answers with speech-to-text. Answers are typed today.
11. More jobs, and admin screens to add or edit job descriptions and question banks.
12. Analytics on score distributions, follow-up rates and token cost per interview.

18. Changes needed for cloud deployment

Data and storage

- Replace the JSON files with a managed database. Several app instances writing to the same JSON files would corrupt the data.
- Store uploaded CVs in encrypted object storage, and keep only what scoring needs.
- Run the retention purge once for the whole system, as a scheduled job.

Timers and in-memory state

- The 60-second time-limit check and the "already scoring" guard live in the memory of one process. With several instances, two instances could score the same interview. Move the guard into the database and run the time-limit check in one place only.

Secrets, network and access

- Keep the OpenAI key and the admin password in a secrets manager, not in a file on the server.
- Set PUBLIC_BASE_URL to the real domain, and NODE_ENV to production.
- Serve everything over HTTPS through a load balancer or reverse proxy, and pass the real client address through.
- Replace the shared admin password with user accounts and single sign-on, with separate roles for admins and reviewers.
- Add rate limits at the edge for the candidate and admin endpoints.

Operations and packaging

- Send logs to a central service, keeping candidate tokens out of them (already done in the code).
- Back up the database, and test that restoring from a backup works.
- Check the data residency and data-processing terms of the OpenAI account before real candidates use the system.
- Add a Dockerfile. At run time the image needs node_modules, dist, public, the job description folder, the question bank folder and package.json.

19. Production readiness plan

The system is a working prototype. It has been run end to end, but it is not yet ready for real candidates. The steps are grouped in the order they should be done.

Step A: before any real candidate uses it

- Legal and policy: review the consent wording, retention period and use of an outside AI service. Sign a data processing agreement with the AI provider. Write a candidate notice. Decide and document who makes the final decision.
- Identity and fairness: add the name check against the CV. Make sure links go only to the named candidate. Run a fuller bias test. Compare the tool with experienced interviewers on past interviews before relying on it.
- Security: replace the shared admin password with named accounts and roles. Move secrets to a secrets manager. Serve over HTTPS only. Add rate limits and CSRF protection.
- Data: move to a database, store CVs encrypted, and make and test backups.

Step B: running it reliably

- One copy at a time, or a database-held lock, so an interview is never scored twice.
- Scheduled cleanup as a job. Monitoring and alerts for failed scoring, failed uploads, OpenAI errors and cost.
- Central logs, a health check that the platform acts on, and a monthly spending cap with the provider.

Step C: improving quality over time

- Keep a set of past interviews with human decisions, and test every change against it.
- Measure score variation regularly. Test injection attempts regularly.
- Review a sample of reports each month, and record where reviewers disagreed with the AI.
- Review each new job set with the hiring manager before use.

Step D: testing before launch

- Automated tests for the main flow: consent, upload, answers, time limit, scoring and review.
- A pilot with a small group of real candidates, with results compared to the usual process.
- An accessibility check, so the candidate pages work with screen readers and on phones.
- A security review by someone who did not build the system.

20. Technology comparison: Node.js vs Python

The main alternative for a project like this is Python with the FastAPI framework. Both have official OpenAI SDKs and both can read PDF and DOCX files. The comparison below is for this project.

Question	Node.js + TypeScript + Express (used here)	Python + FastAPI
Main job of the server	Calls the OpenAI API, reads and writes JSON, serves pages. Mostly waits on the network.	Same
Handling many waiting requests	Non-blocking by default. One process can wait on many model calls.	Non-blocking with async, but blocking code stalls the server.
Language shared with the browser	Yes: the pages are JavaScript, and types can be shared.	No: the team would still need JavaScript for the pages.
Type checking	TypeScript checks types before the code runs.	Type hints are optional; Pydantic adds run-time checks.
AI and ML libraries	Good support for hosted AI APIs. Few local ML libraries.	Strongest ecosystem for local models, embeddings and data work.
CV and document parsing	pdf-parse and mammoth work well for text extraction.	pypdf, pdfplumber and python-docx are mature.
PDF reports	pdfkit (used here).	reportlab, WeasyPrint.
Testing	Jest (used here).	pytest.
Deployment	Small container, one process.	Also simple; needs dependency management.
Best fit when	The product is a web service around hosted AI APIs, with a JavaScript front end.	The product needs local ML models, heavy data work or a data-science team.

Why Node.js for this project

- The work is web requests, JSON and calls to a hosted AI API. Node.js handles that naturally, and waiting on many model calls at once is what it is built for.
- TypeScript keeps the shared records (sessions, questions, scores) consistent.
- One language across the server and the pages keeps the project small.

When Python would be the better choice

- If the app must run its own models (for example local speech-to-text or embeddings) or analyse CVs heavily.
- If the team maintaining the project is mainly data scientists.

Cost of switching: the server code would be rewritten. The data formats, prompts and front-end pages would carry over, so the rewrite is smaller than it sounds. No speed or cost benchmark has been run for the two stacks.

21. Open decisions and next steps

Decisions needed

- [ ] Approve the fix for defect D1 (retry an invalid skill score once, then mark the interview as errored instead of saving NaN).
- [ ] Choose the next guardrails to build: token budget per interview, rate limiting, CSRF protection, bias check, identity check.
- [ ] Record the reviewer's own recommendations for the five evaluation interviews (section 11.4).
- [ ] Decide whether to delete the nine evaluation interview records now, or let the 30-day purge remove them.
- [ ] Decide whether to run Mode C tests on the newest job profiles.

Next steps once decisions are made

1. Make the D1 fix and rerun the scoring consistency test, with at least 20 runs per transcript.
2. Complete the human agreement comparison and explain each disagreement.
3. Add the identity check and the CV-text pattern rules.
4. Start the production readiness plan, Step A.

Appendix A: project documents

File	Purpose
README.md	Full project description: features, stack, structure, architecture, flow, guardrails, tokens, setup, roadmap
docs/SIMPLE_GUIDE.md	Six plain-language answers: what, why, when, how, what can go wrong, production
docs/EVALUATION_REPORT.md	Evaluation: consistency, bias, injection, human agreement, defects, and interview appendix
docs/evaluation/	Raw evaluation results (JSON), the synthetic transcripts and the evaluation runner
docs/PROJECT_OVERVIEW.md	Detailed description of the project
docs/TRACE_FIRST_CLICK.md	Step-by-step trace from a button click to a database record
docs/LEARNING_GUIDE.md	Learning path through the code
docs/INTERVIEW_QA.md	Questions and answers on design and implementation
docs/Project_Report.pdf	This consolidated report

Appendix B: test history

These checks were run during development with temporary scripts kept outside the repository. They were reported at the time of testing. They were not rerun for this report, and some were run before later code changes.

Test (script name)	Result at the time	Notes
Phase 1 (phase1-test.js)	21 of 21 passed	Offline suite
Phase 2 (phase2-test.js)	28 of 28 passed	Offline suite
Phase 3 (phase3-test2.js)	21 of 21 passed	Offline suite
CV extraction (cv-test.js)	11 of 11 passed	Offline suite
Live link (link-live.js)	33 of 33 passed	Run against the running server
Timer (timer-live.js)	6 of 6 passed	Run against the running server
Token measurement (tokens.js)	One run per mode	Results in section 10
Mode C live run on new roles	Not run	Cancelled before completion; not run again as instructed